

CEG 4166 / CSI 4141 – Real-Time Systems Design
SEG 4145 – Real-Time and Embedded Software Design
Winter 2026

Laboratory 3 – User Interaction and Synchronization

Extension of Laboratory 2

Period: February 17 → March 7

Weight: 20%

1. Context and Pedagogical Continuity

Laboratory 3 builds directly on the FreeRTOS architecture developed in Laboratory 2.

In Lab 2:

- Motor control was open-loop.
- UI polling was done inside a periodic UI Task.
- State transitions were triggered by button polling.
- Architecture used two tasks (Motor + UI).

In Lab 3:

- User interaction becomes interrupt-driven.
- Synchronization between ISRs and tasks is introduced.
- State consistency becomes critical.
- LCD and touch keypad are integrated.
- The architecture must be made scalable for future labs (closed-loop control).

No super-loop designs are allowed.

The Lab 2 architecture must be reused and extended — not rewritten.

2. Learning Objectives

By the end of this laboratory, students will be able to:

- Convert polling-based UI into interrupt-driven architecture
- Safely synchronize ISRs and FreeRTOS tasks
- Guarantee deterministic motor behavior under concurrent events
- Extend an embedded architecture without breaking modularity
- Implement state-machine-based control

3. Hardware Configuration (Mandatory)

All Lab 2 hardware remains active.

3.1 Motor Driver (unchanged from Lab 2)

Function	Pin
Standby (STB)	PE15
PWM (TIM1_CH1)	PE9
AI1	PB10
AI2	PB11

3.2 Potentiometer (unchanged)

Function	Pin
ADC input	PC0

3.3 LEDs

Color	Pin
Red	PA9
Blue	PB7
Green	PC7

3.4 Buzzer

Function	Pin
Buzzer	PA0

Must be non-blocking.

3.5 User Interaction Inputs (NEW in Lab 3)

Push Button (Interrupt-Based)

Function	Pin
Blue Button	PC13 (EXTI)

Must trigger state cycling using an interrupt.

Touch Keypad (I2C + Interrupt)

Function	Pin
Touch Interrupt	PB6
Touch Enable	PB5
I2C SDA	PF0
I2C SCL	PF1

Touch must implement:

- **STOP** (Motor stop)
- **CW** (Clockwise motor rotation)
- **CCW** (Counter-Clockwise motor rotation)

Emergency Stop (Optional but recommended)

Function	Pin
E-Stop	PB2

Must trigger a highest-priority STOP event. The motor must be disabled within one Motor Task period (≤ 20 ms).

4. System Behavior Requirements

4.1 Initial State (Same as Lab 2 – Mandatory)

At reset:

- State = STOP
- PWM = 0
- Red LED ON

- Motor not rotating

4.2 State Machine (Central Requirement)

The system must use a centralized state machine.

Valid states:

State	Motor	LED
STOP	Disabled	Red
CW	Clockwise	Green
CCW	Counter-clockwise	Blue

Only one LED ON at any time.

4.3 Push Button Behavior

Each press cycles:

STOP → CW → CCW → STOP

Must:

- Be interrupt-driven
- Send event to RTOS task
- Generate short beep
- Update LCD

4.4 Touch Keypad Behavior (UPDATED – Required Mapping)

The touch keypad (capacitive touchpad) must be used as a keypad input device. The touch sensor generates an interrupt pulse on **PB6** whenever a touch event occurs. The application must then read the touch sensor status via **I2C (PF0/PF1)** to determine which key was pressed.

Mandatory key mapping for Lab 3:

- **Key “1” → CW**
- **Key “2” → CCW**
- **Key “0” → STOP**

Rules:

1. Touch commands **override** the cyclic order from the Blue Button.
2. Each valid touch command must:
 - Update the centralized system state
 - Produce a confirmation beep (non-blocking)
 - Update the LCD display immediately
3. The PB6 interrupt ISR must **not** read I2C and must **not** directly change motor state. It must only notify the RTOS (task notification / queue / event group).

Multi-touch handling (mandatory policy):

- If multiple keys are detected simultaneously, the system must apply a deterministic priority:
 - **STOP (0) > CCW (2) > CW (1)**
(i.e., STOP always wins.)

4.5 LCD Display Requirements (NEW – Mandatory)

The LCD must display the **current motor state at all times**.

The display must show one of the following values:

- **STOP**
- **CW**
- **CCW**

The LCD driver is provided to students as **lcd.h** and **lcd.c**. Students must include these files in their Lab 3 project and use the driver to implement the required LCD behavior.

Integration rules:

1. **LCD initialization must be performed once at startup** (before the FreeRTOS scheduler starts).
2. **LCD updates must be performed only inside the Control/State Task**, not inside any ISR.
3. The LCD must be updated **immediately after any valid state change**.
4. The LCD must always match the real motor state (**no mismatch allowed**).
5. **No blocking delays** are allowed during LCD updates.

Example Behavior

Event	LCD Display
System reset	STOP
Touch “1”	CW
Touch “2”	CCW
Touch “0”	STOP
Button cycle	Updated accordingly

The LCD display is part of system consistency and will be evaluated during demonstration.

Requirements

1. The LCD must be updated immediately after any valid state change.
2. The displayed value must always match the actual motor state.
3. The LCD update must be handled inside the **Control/State Task**, not inside an ISR.
4. The LCD must never display an outdated state.
5. No blocking delays are allowed during LCD updates.

Example Behavior

Event	LCD Display
System reset	STOP
Touch “1”	CW
Touch “2”	CCW
Touch “0”	STOP
Button cycle	Updated accordingly

The LCD display is part of system consistency and will be evaluated during demonstration.

4.6 Buzzer Behavior

The buzzer is used to confirm that events are correctly propagated through the RTOS architecture.

Must generate:

- Short beep on state change (30–80 ms recommended)
- Confirmation beep on keypad command

Must be non-blocking.

HAL_Delay() inside tasks is forbidden.

5. Required FreeRTOS Architecture (Updated)

Lab 2 had:

- Motor Task (High priority)
- UI Task (Lower priority)

Lab 3 must evolve to:

5.1 Motor Task (Highest Priority – unchanged)

Responsible for:

- Applying state
- Applying PWM
- Enforcing safety
- Updating LEDs

Period: 10–20 ms

Motor Task must be the only task allowed to modify PWM and motor direction GPIO signals.

5.2 Control/State Task (NEW)

Responsible for:

- Receiving events from ISRs
- Updating system state
- Sending commands to Motor Task
- Triggering buzzer
- Updating LCD
- The Control/State Task is the only task allowed to update the LCD using the provided lcd.h/lcd.c driver

This task becomes the central decision layer.

5.3 ADC Task (Optional Separation)

May remain inside UI task or separated.

5.4 Interrupt Service Routines (NEW)

ISR must:

- NOT change motor state directly
- NOT control hardware directly

- Send event using:
 - xTaskNotifyFromISR()
 - OR queue
 - OR event group

Proper ISR-to-task synchronization is mandatory.

6. Synchronization Requirements (Critical)

Students must demonstrate:

- Safe communication between ISR and tasks
- No race conditions
- Deterministic behavior
- Motor always safe in STOP

Improper synchronization results in penalty.

7. Required In-Lab Demonstration

Students must show:

1. FreeRTOS scheduler running
2. At least 3 logical components:
 - Motor control
 - Event handling
 - User interaction
3. Interrupt-driven button
4. Touch keypad commands
5. LCD updating state
6. Buzzer feedback
7. Deterministic behavior under rapid input

8. Demonstrate touch keypad mapping:

- Touch “1” → motor rotates **CW**
- Touch “2” → motor rotates **CCW**
- Touch “0” → motor **stops immediately** (PWM = 0)

Instructor may test rapid consecutive touches to verify determinism and synchronization.

8. Deliverables

8.1 Functional Code

- Extension of Lab 2 project
- Clean modular structure
- No blocking delays
- Clear separation of ISR and tasks
- lcd.h and lcd.c integrated and used to display STOP/CW/CCW

8.2 Updated Architecture Diagram

Must show:

- ISR layer
- Event mechanism
- Control/State Task
- Motor Task
- Hardware interface

8.3 Synchronization Explanation (Short)

Students must explain:

- Why ISRs cannot directly change state

- Why Motor Task has highest priority
- How determinism is preserved

9. Evaluation Rubric – Lab 3 (20 pts)

Criterion	Points
Interrupt-based state sequencing	4
LED and state consistency	4
Touch keypad integration	5
LCD state display and consistency with motor state	2
Buzzer behavior	3
RTOS synchronization correctness	2
Total	20

10. Instructor Note

This lab prepares students for:

- Closed-loop control (Lab 4)
- Encoder feedback
- PID control
- Real-time constraints
- Event-driven architectures